
CS 61B

Fall 2024

Midterm 1

Friday September 27, 2024

Your Name: _____

Your SID: _____ Location: _____

SID of person to your left: _____ Right: _____

This exam is worth **100 points**, and consists of 8 questions. Questions are ordered by topic, not by difficulty.

Question:	1	2	3	4	5	6	7	Total
Points:	8	9	6	18	15	34	10	100

- ☐ indicates only one circle should be filled in. ☐ indicates more than one box may be filled in. **Please fill in the shape completely.** If you change your response, **erase as completely as possible.**
- Anything you write that you ~~cross out~~ will not be graded.
- If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade according to the worst interpretation.
- Unless otherwise specified, all data structures and algorithms behave according to their implementation in lecture, with no additional optimizations.
- If an implementation detail (e.g. tiebreaking scheme, linked list topology) is relevant, it will be explicitly noted in the question.

Coding questions:

- You may write at most one statement per blank and you may not use more blanks than provided.
- Your answer will be reformatted according to the 61B style guidelines. For example, any if-statement requires at least three lines for the purposes of determining line count.
- You may not use ternary operators, lambdas, streams, or multiple assignment.
- Unless otherwise specified, you may assume that everything on the reference card has been imported.

Write the statement below in the same handwriting you will use on the rest of the exam. Failure to do so may result in a voided exam.

I have neither given nor received help on this exam (or quiz), and have rejected any attempt to cheat. If these answers are not my own work, I may be deducted up to 9,876,543,210 points.

Signature: _____

1 Football?

(8 Points)

For each of the following lines, write the result of each statement: Write “CE” if a compiler error is raised on that line, “RE” if a runtime error occurs on that line, “OK” if the line runs properly but doesn’t print anything, and the printed result if the line runs properly and prints something. If a line errors, assume that the program continues to run as if that line did not exist. Blank lines will receive no credit.

```
public class Athlete {
    public void g() {
        Athlete curr = (FootballPlayer) this;
        p(curr);
    }
    public void p(Athlete curr) {
        curr.celebration();
    }
    public void celebration() {
        System.out.println("Let's go");
    }
    public Athlete spawn() {
        System.out.println("spawn Athlete");
        return new Athlete();
    }
}

public class FootballPlayer extends Athlete {
    @Override
    public void celebration() {
        System.out.println("Let's goooal");
    }
    @Override
    public Athlete spawn() {
        System.out.println("spawn Footballer");
        return new FootballPlayer();
    }
}
```

(a) (6 pts)

```
public class Main {
    public static void main(String[] args) {
        Athlete messi = new FootballPlayer();
        FootballPlayer brady = new Athlete();
        Athlete thiago = messi.spawn();
        FootballPlayer lionel = (FootballPlayer) messi;
        messi.g();
        thiago.g();
    }
}
```

	1
	2
	3
	4
	5
	6

(b) (2 pts) Which of these method signatures would, if added by itself, allow `messi.f(messi)` to compile? Select all that apply.

- ☐ `public void f(FootballPlayer x)`, added to class `Athlete`.
- ☐ `public void f(FootballPlayer x)`, added to class `FootballPlayer`.
- ☐ `public void f(Athlete x)`, added to class `Athlete`.
- ☐ `public void f(Athlete x)`, added to class `FootballPlayer`.

2 Potpourri

(9 Points)

- (a) (1.5 pts) Which of the following is a valid feature of a Java interface? Select all that apply.
- ☐ Private methods
 - ☐ Public methods
 - ☐ Constructors
- (b) (1 pt) True or False: A class can implement multiple interfaces.
- ☐ True
 - ☐ False
- (c) (1 pt) Which statement best describes the test-driven development cycle?
- ☐ Write tests → Refactor code → Write code
 - ☐ Write tests → Write code → Refactor code
 - ☐ Write code → Refactor code → Write tests
- (d) (1 pt) What is the primary purpose of the “Refactor” phase in test-driven development?
- ☐ To reduce code complexity and improve design while keeping the tests passing.
 - ☐ To rewrite test cases that are failing.
 - ☐ To add additional features to the code.
- (e) (2.5 pts) Which statements about access modifiers are true? Select all that apply.
- ☐ If a public method relies on a helper function to work, the helper function must be public.
 - ☐ If a private method relies on a helper function to work, the helper function must be private.
 - ☐ Private methods and variables can be accessed by subclasses.
 - ☐ Protected methods and variables can be accessed by subclasses.
 - ☐ The value of a private variable can be accessed via a public method.
 - ☐ None of the above

(f) (1 pt) What is the result of running `main` below?

```
public static <T> void swap(T[] array) {  
    T temp = array[0];  
    array[0] = array[1];  
    array[1] = temp;  
}  
public static void main(String[] args) {  
    Integer[] values = new Integer[]{10, 20};  
    swap(values);  
    System.out.println(values[0]);  
}
```

☐ 10

☐ 20

(g) (1 pt) What is the result of running `main` below?

```
public static <T> void swap(T x, T y) {  
    T temp = y;  
    y = x;  
    x = temp;  
}  
public static void main(String[] args) {  
    Integer x = 10;  
    Integer y = 20;  
    swap(x, y);  
    System.out.println(x);  
}
```

☐ 10

☐ 20

3 What Even Is Static?

(6 Points)

```
public class Q3 {
    private int instanceVar = 0;
    private static int staticVar = 1;

    public void instanceMethod() {
        // Code here for part a
    }

    public static void staticMethod() {
        // Code here for part b
    }

    public static void main(String[] args) {
        Q3 obj = new Q3();
        // Code here for part c
    }
}
```

- (a) (1.5 pts) Select all lines of code that, if added to `instanceMethod` by itself, would compile without error.
- ☐ `System.out.println(instanceVar);`
 - ☐ `System.out.println(staticVar);`
 - ☐ `staticMethod();`
- (b) (1.5 pts) Select all lines of code that, if added to `staticMethod` by itself, would compile without error.
- ☐ `System.out.println(instanceVar);`
 - ☐ `System.out.println(staticVar);`
 - ☐ `instanceMethod();`
- (c) (3 pts) Select all lines of code that, if added to the end of the `main` method by itself, would compile without error.
- ☐ `Q3.instanceMethod();`
 - ☐ `Q3.staticMethod();`
 - ☐ `obj.instanceMethod();`

4 Crossword

(18 Points)

Daniel is working on completing a crossword puzzle, when he realizes he can represent it as a matrix! He creates a N by N 2D array of characters, with **black spaces denoted by hashtags (#)**.

For example, consider the following 11-by-11 crossword. In the first row, “MADAM” and “PEA” are horizontal words. In the first column, “MUTE” and “TOP” are vertical words.

#	#	#	#	#	#	#	#	#	#	#
#	M	A	D	A	M	#	P	E	A	#
#	U	V	U	L	A	#	O	R	C	#
#	T	O	K	E	N	#	L	E	T	#
#	E	W	E	#	D	U	O	#	#	#
#	#	#	#	M	A	P	#	#	#	#
#	#	#	O	A	T	#	L	A	P	#
#	T	A	X	#	O	Z	O	N	E	#
#	O	R	E	#	R	O	U	T	E	#
#	P	E	N	#	Y	O	D	E	L	#
#	#	#	#	#	#	#	#	#	#	#

- (a) (3 pts) Daniel decides to first make a helper function that computes the transpose of the crossword.

The **transpose** operation flips the matrix over its diagonal, such that the element at position (i, j) in the original matrix becomes the element at position (j, i) in the transposed matrix.

For example, the transpose of:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \quad \text{is:} \quad \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$$

Complete the implementation of `transpose`, which receives as input a matrix, and returns a new matrix that is the transpose of the input matrix.

```
private static char[][] transpose(char[][] matrix) {
    int n = matrix.length;

    char[][] result = _____;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            _____;
        }
    }
    return result;
}
```

- (b) (15 pts) Complete the implementation of `validCrossword`, which receives as input a crossword puzzle, and determines if all horizontal and vertical words in the crossword puzzle are valid according to `validWord`. You may assume that the outer border of the crossword is composed entirely of “#”s.

You have access to `transpose`, as well as the `StringBuilder` class and the `validWord` method:

```
public final class StringBuilder {
    /** Appends the specified char to this StringBuilder. */
    public void append(char c) {...}

    /** Outputs the String under construction. */
    public String toString() {...}
}

/** Returns whether word is a valid word. Returns false on an empty input string. */
public static boolean validWord(String word) { ... }

public static boolean validCrossword(char[][] crossword) {

    return _____;
}

private static boolean verifyRows(char[][] crossword) {
    int n = crossword.length;
    for (int i = 0; i < n; i++) {
        StringBuilder word = new StringBuilder(); // Resets word to empty string

        for (_____ : _____) {

            if (_____) {

                _____;
            } else {
                String s = _____;

                if (_____ && _____) {

                    return _____;
                }

                _____;
            }
        }
    }
    return _____;
}
```

5 Tiebreakers

(15 Points)

In many competitions where ties are possible, we use multiple tiebreakers to determine a winner, where each subsequent tiebreaker is applied only if the previous one results in a tie.

Implement `TiebreakingComparator<T>`, whose constructor takes in a `List` of `Comparators`, and behaves as follows:

- If a `Tiebreaking Comparator` is given two objects, then they are compared according to the first `Comparator` in its input list.
- If the result is equality (the two objects are equal under the first `Comparator`), they are then compared using the second `Comparator` in the list. If the result is again equality, they are then compared using the third `Comparator`, and so on.
- This continues until either a nonequal comparison result occurs (in which case the `Tiebreaking Comparator` returns that result), or all `Comparators` in the list are used (in which case the `Tiebreaking Comparator` returns equality).
- **You may not call Comparators more times than is necessary.** For example, you may not call a `Comparator` twice, and you may not call the second `Comparator` if the first `Comparator` would have yielded a result.

```
public class TiebreakingComparator<T> implements Comparator<T> {

    _____ 1 _____ lst;

    public TiebreakingComparator (_____ 2 _____ lst) {

        _____ 3 _____;

    }
    @Override
    public int compare(T a, T b) {

        for(_____ 4 _____ : _____ 5 _____) {

            _____ 6 _____;

            _____ 7 _____ {

                _____ 8 _____;

            }

        }

        _____ 9 _____;

    }
}
```

6 Linked Links

(34 Points)

The `Ordinal` class is similar to a Linked List, except its nodes do not contain items; each node in an `Ordinal` contains a pointer to another `Ordinal`, and nothing else.

```
public class Ordinal {
    public Ordinal rest;

    public Ordinal(Ordinal rest) {
        this.rest = rest;
    }
}
```

Despite being seemingly useless, it is possible to use this class to represent an extension of the nonnegative integers called the *Ordinal Numbers*.

We will use *italics* to signify ordinal numbers. For example, 0 is the ordinal number representing 0.

- $0 = \text{null}$
- $1 = \text{Ordinal}(\text{null})$
- $2 = \text{Ordinal}(\text{Ordinal}(\text{null})) = \text{Ordinal}(1)$
- In general, $n = \text{Ordinal}(n - 1)$.

In this question, you may assume that all input `Ordinals` correspond to some nonnegative integer n .

- (a) (2 pts) Write a method `succ`, which receives as input an `Ordinal` representing n , and returns an `Ordinal` representing $n + 1$ (the *successor* of n).

```
Ordinal succ(Ordinal curr) {

    return _____;
}
```

- (b) (2 pts) Write a method `pred`, which receives as input an `Ordinal` representing n , and returns an `Ordinal` representing $n - 1$ (the *predecessor* of n).

```
Ordinal pred(Ordinal curr) {
    if(curr == null) {
        throw new java.lang.IllegalArgumentException();
    }

    return _____;
}
```

For the remainder of the question, you may use `prev` and `succ` in your answers.

- (c) (6 pts) Write a method `add`, which receives as input two `Ordinal`s representing a and b , and returns an `Ordinal` representing $a + b$.

```
Ordinal add(Ordinal a, Ordinal b) {

    while (_____1_____) {

        _____2_____;

        _____3_____;

    }

    return _____4_____;

}
```

- (d) (14 pts) Write a method `floordiv`, which receives as input two `Ordinal`s representing a and b , and returns an `Ordinal` representing a/b , rounding down.

```
Ordinal floordiv(Ordinal a, Ordinal b) {

    Ordinal result = _____1_____;
    final Ordinal bcopy = b;

    while (_____2_____) {

        _____3_____;

        _____4_____;

        if (_____5_____) {

            _____6_____;

            _____7_____;

        }

    }

    return _____8_____;

}
```

- (e) (10 pts) Rewrite the class `Ordinal` to implement `Iterable`. The iterator over the `Ordinal` n returns, in order, $n - 1$, $n - 2$, $\dots$, 2 , 1 , 0 . You may assume that $n > 0$.

Note that n is not returned by the iterator.

```
public class Ordinal implements Iterable<Ordinal> {
    Ordinal rest;

    public Ordinal (Ordinal rest) {
        this.rest = rest;
    }

    public Iterator<Ordinal> iterator() {

        return _____;
    }

    private class OrdinalIterator _____ {
        Ordinal curr;
        public OrdinalIterator(Ordinal curr) {

            _____;
        }

        @Override
        public boolean hasNext() {

            return _____;
        }

        @Override
        public Ordinal next() {
            if (curr == null) {
                throw new java.util.NoSuchElementException();
            }

            _____;

            return _____;
        }
    }
}
```

7 Infinite Linked Links

(10 Points)

Note: This problem is a continuation of the previous problem, titled “Linked Links”. Assume that all parts from “Linked Links” are correctly implemented.

The Ordinal class is reprinted below.

```
public class Ordinal {
    public Ordinal rest;

    public Ordinal(Ordinal rest) {
        this.rest = rest;
    }
}
```

The Ordinal ω is a representation of (the smallest) infinity in the ordinal numbers, and can be constructed by the following code:

```
public static Ordinal inf() {
    Ordinal omega = new Ordinal(null);
    omega.rest = omega;
    return omega;
}
```

Unlike in the nonnegative integers, it is possible to count *beyond* ω , as follows:

- ω is defined as above.
- $1 + \omega = \text{Ordinal}(\omega)$
- $2 + \omega = \text{Ordinal}(1 + \omega)$
- In general, $n + \omega = \text{Ordinal}(n - 1 + \omega)$ for all integers $n > 0$.

We will refer to any ordinal that can be constructed by the above process as an **infinite ordinal**, and any ordinal corresponding to a nonnegative integer (as in the previous question) as a **finite ordinal**.

- (a) (1 pt) True or False: It is still possible to construct ω (potentially by modifying `inf`) if `rest` is **final**, and no other changes are made to the Ordinal class.

- ☐ True
☐ False

- (b) (1 pt) True or False: `succ( $\omega$ ) ==  $\omega$`

- ☐ True
☐ False

- (c) (1 pt) True or False: `pred( $\omega$ ) ==  $\omega$`

- ☐ True
☐ False

- (d) (3 pts) Which of the following can occur if `add` is called with at least one infinite ordinal? Select all that apply.

- ☐ `add` returns an infinite ordinal.
- ☐ `add` returns a finite ordinal.
- ☐ `add` infinite-loops or crashes.

- (e) (1 pt) True or False: `add(a, b)` will always exhibit the same behavior as `add(b, a)`.

In other words, if `add(a, b)` infinite-loops/crashes, then `add(b, a)` will also infinite-loop/crash, and if `add(a, b)` returns a value, then `add(b, a)` will also return the same value.

- ☐ True
- ☐ False

- (f) (1 pt) True or False: `floordiv( $\omega$ , b)` will return ω if `b` is a finite ordinal.

- ☐ True
- ☐ False

- (g) (1 pt) True or False: `floordiv(a,  $\omega$ )` will return 0 if `a` is a finite ordinal.

- ☐ True
- ☐ False

- (h) (1 pt) What is the result of the expression `inf() == inf()`?

- ☐ `true`
- ☐ `false`

Nothing on this page is worth any points.

Bonus Question

(0 Points)

Write a crossword clue for one of the words in the example crossword in Question 4. We will compile the best clues for each word. Cryptic clues and Printer's Devilry will also be accepted.



Feedback

(0 Points)

Leave any feedback, comments, concerns, or more drawings below!

